

Spring Boot

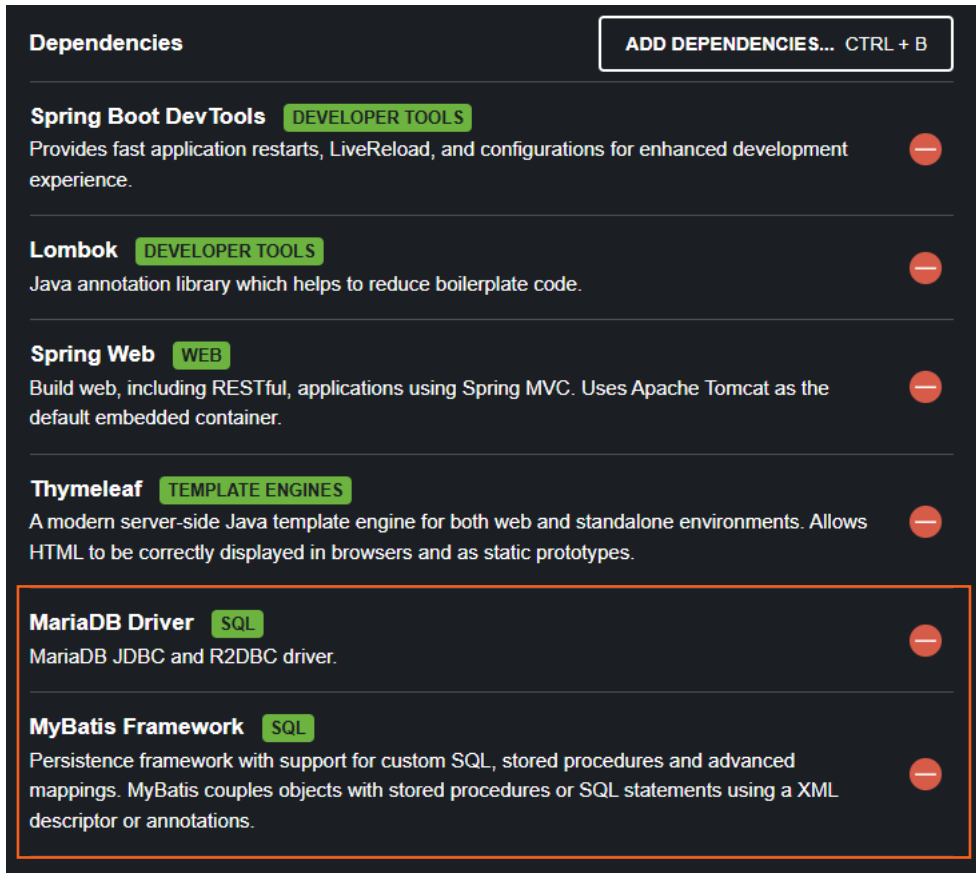
Part5

*프로젝트에 데이터베이스 연동 설정 하기
게시글 목록 조회 기능 구현하기
게시글 상세정보 기능 구현하기
게시글 등록 기능 구현하기*

시작하기전에

Spring과 데이터베이스를 연동하여 기능을 구현하는 것이 웹 개발의 핵심입니다. 하지만 막상 수업을 들으면 다수의 파일이 서로 연관되어 있기 때문에 상당히 복잡하고 어렵게 느껴집니다. 하지만 2~3번 반복해보면 계속 같은 코드만 작성한다고 느껴질 겁니다. 진입 장벽은 높지만 데이터베이스 관련 기능을 구현하는 코드는 항상 비슷한 패턴을 가지기 때문입니다. 계속 반복하세요. 나중에는 웹 개발에서 DB 기능을 구현하는 것이 가장 쉬운 부분 중 하나로 느껴질 정도로 똑같은 패턴만 사용합니다. 응원합니다!!!

Spring에서 데이터베이스 기능을 구현하기 위해 우선 프로젝트 생성 시 MariaDB Driver와 Mybatis Framework를 의존성 추가한다.



- MariaDB Driver
Spring과 MariaDB를 연동할 수 있게 해주는 라이브러리
 - MyBatis Framework
Spring에서 데이터베이스 기능을 구현할 때 많이 사용하는 Framework.
스프링 자체에도 db기능 구현을 위한 메서드가 존재하지만 사용하지 않음
 - 프로젝트 생성 시 스프링 버전이 4.1 이전 버전을 사용한다.
4.1 이후 버전은 mybatis를 지원하지 않는다.
- DB 연동을 위해 위 2개의 의존성을 추가해서 생성한 프로젝트는 실행하면 오류가 발생한다. DB연동을 사용하기 위해 의존성을 추가하면 반드시 DB 연동을 해주어야 하는데, 프로젝트 생성 직후 실행할 때는 DB 연동 코드를 작성하지 않았기 때문이다. 이 오류는 DB 연동 코드를 작성하면 자연스럽게 사라지게 된다.

실무에서 데이터베이스 기능 구현을 위해 많이 사용하는 것은 Mybatis와 JPA가 있다. 둘 중 JPA의 사용률이 점점 올라가고 있으며, 메커니즘도 Mybatis보다 웹 개발에 더 적합하지만 개인적으로 초보자가 공부하기에는 적합하지 않다고 판단한다. 그 이유는 JPA는 쿼리문도 자동으로 만들어주기 때문에, 쿼리문에 익숙하지 않은 우리가 사용하기에는 생략된 내용이 너무 많아 이해하기 어렵기 때문이다.

프로젝트 생성 후 기본적인 세팅을 끝내면 Spring과 DB를 연동하는 설정을 해야한다.

이 설정은 src/main/resources 폴더 안의 `application.yaml` 파일에 작성한다.

DB 연동 코드는 정해져 있기 때문에 공유한 txt파일의 모든 내용을 복사하여 `application.properties` 파일에 붙여넣기한다.

build.gradle 파일의 `dependencies {}` 안에

```
implementation 'org.bgee.log4jdbc-log4j2:log4jdbc-log4j2-jdbc4:1.16'
```

코드를 추가한다. 이 코드는 실행되는 쿼리를 콘솔창에 로그로 보여주는 라이브러리를 프로젝트에 추가한 것이다.

마찬가지로, `implementation("org.mybatis:mybatis-typehandlers-jsr310:1.0.2")` 코드도 추가한다. 해당 코드는 Mariasb의 Datetime 날짜 데이터를 자바의 LocalDateTime 자료형으로 데이터를 받을 수 있게 해준다.

build.gradle 파일의 `내용이 변경되면 반드시 코끼리 버튼을 클릭`해야 적용된다.

src/main/resources 폴더에 공유한 `logback.xml`, `log4jdbc.log4j2.yaml` 파일을 붙여넣기한다.

이 두 파일은 실행되는 쿼리문이 콘솔창에 로그로 표현되는 방식을 지정하는 여러 설정이 들어있는 파일들이다. 그냥 붙여넣기하여 사용하면 된다.

src/main/resources 폴더 안에 `mapper`라는 이름으로 폴더를 생성하고, mapper 폴더 안에 공유한 `mapper.xml` 파일을 붙여넣기한다.

`mapper.xml` 파일에는 실행할 쿼리문을 작성한다.

본격적으로 게시글 CRUD 기능을 코드로 작성하며, 데이터베이스 기능을 구현해보겠다.

데이터베이스와 연관된 모든 코드 및 파일은 데이터베이스의 테이블을 기준으로 구현된다. 그렇기에 이번 게시글 CRUD 기능 실습을 위한 테이블을 다음 쿼리문을 통해 만들어주자.

```
CREATE TABLE BASIC_BOARD (  
    BOARD_NUM INT PRIMARY KEY AUTO_INCREMENT #컬럼의 값을 지정하지 않으면 자동으로 1씩 증가된 값을 넣어주는 기능을 한다.  
    , TITLE VARCHAR(30) (최대 30글자)  
    , WRITER VARCHAR(20)  
    , CONTENT VARCHAR(50)  
    , READ_CNT INT DEFAULT 0 정수 입력 가능하지만 입력값 없으면 0으로 입력  
    , CREATE_DATE DATETIME DEFAULT SYSDATE()  
);
```

위에서도 언급했지만 SPRING에서 데이터베이스와 연동된 기능을 구현할 때, 모든 코드는 생성된 테이블을 기준으로 작성된다. 그렇기 때문에 웹을 구현할 때 데이터베이스를 모르면 어떠한 코드도 작성할 수 없다. 데이터베이스는 아주 중요하다.

CRUD란 C(create/등록), R(Read/조회), U(update/수정), D(delete/삭제) 기능을 말하며, 모든 프로그램의 기본 기능을 의미한다.

이러한 기능은 데이터베이스와도 함께 짝지을 수 있는데, C – INSERT, R – SELECT, U – UPDATE, D – DELETE 쿼리문으로 연결해서 생각할 수 있다.

다음은 게시글 CRUD 기능 구현을 위해 필요한 파일이다.

- resources/mapper/board-mapper.xml
-> 실제 실행할 모든 쿼리문을 작성할 파일.
- com.green.board.mapper.BoardMapper (interface)
-> board-mapper.xml 파일과 연결되어 board-mapper.xml 파일에 작성한 쿼리문을 실행시킬 추상메서드를 정의하는 인터페이스
- com.green.board.dto.BoardDTO
-> BASIC_BOARD 테이블의 데이터를 주고 받을 수 있는 클래스
- com.green.board.service.BoardService
-> BoardMapper 인터페이스에 정의된 쿼리 실행 추상메서드를 활용하여 실제 기능을 구현하는 클래스
- com.green.board.controller.BoardController
-> 구현한 모든 기능을 실행하는 컨트롤러

```
@Setter
@Getter
@ToString
public class BoardDTO {
    private int boardNum;
    private String title;
    private String writer;
    private String content;
    private int readCnt;
    private Timestamp createDate;
}
```

- com.green.board.dto 패키지에 BoardDTO 클래스를 생성한다.
- DTO(Data Transfer Object)는 '데이터 전송 객체'라는 의미로 BASIC_BOARD 테이블과 데이터를 주고 받을 수 있는 클래스를 만드는 것이다.
- 해당 클래스는 테이블의 각 컬럼에 매칭되는 멤버변수를 만들어주면 된다. 이때 멤버변수명은 반드시 테이블의 컬럼명을 기준으로 작성해야 하며, 컬럼명에서 언더바를 제외하고 언더바 뒤의 글자는 대문자로 작성하면 된다.
- 테이블의 각 컬럼과 매칭되는 멤버변수 생성 후 반드시 모든 멤버변수에 대한 getter와 setter를 만들어 준다.
- toString 메서드의 오버라이딩은 필수는 아니지만, 만들어놓으면 개발 시 데이터 정보를 확인할 수 있어 편하다.
- 데이터베이스의 DATETIME 자료형은 자바에서 java.sql.Timestamp로 선언한다.
- BoardDTO 객체 하나는 BASIC_BOARD 테이블에서 하나의 행 데이터를 저장할 수 있는 자료형이다.

BOARD_NUM	TITLE	WRITER	CONTENT	READ_CNT	CREATE_DATE
1	test1	writer	content1	0	2025-01-31 20:02:01
2	test2	writer2	content2	0	2025-01-31 22:36:15
3	test3	writer3	content3	0	2025-01-31 22:37:17

하나의 행 데이터는 BoardDTO 객체 하나와 매칭 됨

```
@Mapper
public interface BoardMapper {
    //게시글 목록 조회
    //게시글 조회
    //게시글 등록
    //게시글 삭제
    //게시글 수정
}
```

BoardMapper 파일

```
<mapper namespace="com.green.board.mapper.BoardMapper">
</mapper>
```

board-mapper.xml파일

실제 실행할 쿼리문을 작성

연동

mappr : 추상메서드 작성

- BoardMapper 인터페이스는 BASIC_BOARD 테이블과 관련된 쿼리문을 실행할 목적의 추상메서드를 작성하는 파일이다.
- 해당 인터페이스가 쿼리문을 실행할 메서드가 있는 인터페이스임을 Mybatis에게 인지시켜 주기 위해 @Mapper 어노테이션을 추가한다.
- 실행할 쿼리문 작성은 공유한 mapper.xml 파일에 작성한다. 인텔리제이 무료 버전에서는 xml 파일 생성이 불편하기 때문에 공유한 파일을 복사하여 사용하도록 한다.
- 공유한 mapper.xml 파일 복사 후 파일명을 적절하게 변경한다. 지금은 게시글 관련 쿼리문을 작성할 것이기에 board-mapper.xml로 변경하였다.
- mapper.xml 파일의 namespace에는 해당 xml 파일과 연결할 인터페이스 파일명을 작성한다. 이때, 반드시 패키지명까지 작성하도록 한다.
- 정리하면 mapper.xml에서 실행할 쿼리문을 작성하고, 해당 xml 파일의 쿼리문을 실행시킬 수 있는 메서드를 interface에서 선언하는 것이다.

그리고 두 파일의 연결을 위해 interface에서 @Mapper 어노테이션을 사용하고, xml 파일의 namespace에서는 연결할 interface 파일명(패키지포함)을 작성하는 것이다.

application.properties 설정 파일에 보면 mybatis.mapper-locations=classpath:mapper/*.xml 이라는 설정 코드가 작성되어 있다.

이 코드는 쿼리문이 작성된 xml 파일의 위치를 지정하는 설정 내용이다. classpath는 resources 파일을 의미하며, '*'는 '모든 이름'을 의미한다.

즉, 해당 설정은 resources 폴더 아래에 존재하는 파일명이 '.xml'로 끝나는 모든 파일을 쿼리문이 작성한 파일임을 인지시키는 역할을 한다.

지금까지가 spring에서 쿼리 실행 기능을 구현하기 위한 기초 작업이다. 아마 벌써 어려움을 느끼고 있을 것이다. 다시 말하지만, 쿼리 실행 기능을 구현하기 위해서는 많이 파일이 필요하고, 그 많은 파일들이 유기적으로 연결되기 때문에 처음보면 상당히 복잡하며, 어려움을 느낄 수 밖에 없다. CRUD 기능을 2~3번 반복해서 만들다보면 동일한 패턴을 가지고 코드가 구현되는 것을 느낄 수 있을 것이다. 그때부터는 쉬워진다. 그러니 너무 스트레스 받지말고 반복적으로 만들어보자. 생각보다 할만 할 것이다.

이제부터 spring에서 쿼리 기능을 실행할 수 있도록 코드를 작성해보겠다. 익숙해지면 여러분이 편한 순서대로 파일을 생성하고 코드를 구현하겠지만, 학습 초반에는 코드를 작성할 파일들이 많아 어떤 파일에서 무슨 코드를 작성해야 할지 막막할 수 있다. 그렇기 때문에 익숙해지기 전까지는 다음의 순서대로 코드를 구현하도록 하자.

1. 기능 구현 혹은 HTML로 화면을 구현할 때 쿼리가 필요한지 파악한다.(쿼리가 필요하다면 2번부터~, 필요없다면 5번,6번만 진행한다)
2. xml 파일에 실행할 쿼리문 작성
3. xml 파일과 연결된 Mapper interface에 1번에서 작성한 쿼리문을 실행시킬 추상메서드 정의
4. Service 클래스에 Mapper Interface에서 정의한 추상메서드를 활용한 핵심기능(SQL를 실행하는 메서드) 구현
5. Controller 클래스에 URL로 들어온 요청을 받아 응답하는 메서드 정의 및 응답할 HTML 파일 생성
6. HTML 화면 구현
7. Service 클래스에서 구현한 메서드를 Controller에서 호출

위 흐름을 보면 알겠지만, 웹 개발 시 대부분의 시작은 쿼리 작성으로 부터 시작한다. 즉, 쿼리문을 작성하지 못한다면 웹 개발을 할 수 없다!

쿼리 공부 열심히 하자!!

참고로 이 pdf 파일에서는 위 절차 중 5번, 6번 절차의 설명은 생략한다.

게시글 목록 조회 및 기본 내용

그럼 앞선 슬라이드에서 설명한 순서대로 게시글 기능을 구현해보자.

우선, 게시글 목록 페이지를 구현할 때 쿼리가 필요한지 파악 한다.(특정 기능 구현 시 쿼리가 필요한지 여부는 웹 개발 공부를 하다보면 자연스레 감이 잡힌다.)

쿼리가 필요한지 파악하기 위해서는 구현 결과 페이지를 떠올려 보면 된다.

No	제 목	작성자	작성일	조회수
6	집에 가고 싶다 ㅏㅏ	홍길동	2024-05-27	5
5	오늘 점심은 중식입니다.	홍길동	2024-05-27	5
4	컴퓨터 포맷 방법 알려주세요.	홍길동	2024-05-27	5
3	우리집 강아지는요...	홍길동	2024-05-27	5
2	이게 맞는거임?	홍길동	2024-05-27	5
1	첫 가입 인사 드려요.	홍길동	2024-05-27	5

우리가 떠올리는 게시글 목록 페이지란 위와 같은 형태를 지니고 있다. 표에 나타나는 게시글 정보는 어디에 저장되어 있을까?

바로 데이터베이스에 저장되어 있다. 결국 데이터베이스에 저장되어 있는 데이터를 화면에 띄우려면 SELECT 쿼리를 이용해 게시글 목록 데이터를 조회하고, 조회한 결과를 HTML파일에 뿌려줘야 하는 것이다. 결국 게시글 목록 페이지를 구현하려면 쿼리 작성이 필요한 것이다.

그러니, 바로 두번째 순서인 쿼리 작성을 진행해보자.

쿼리문 작성은 resources/mapper 폴더 안에 위치한 xml 파일에 작성한다.

게시글 목록 조회 쿼리문을 xml 파일에 작성하기 전에 xml 파일 작성에 대한 기초를 먼저 알아보자.

- 실행할 모든 쿼리문은 XML 파일의 `<mapper></mapper>` 태그 안에 작성한다.
- `SELECT` 쿼리문은 `<select></select>` 태그 안에 작성한다.
- 마찬가지로 `INSERT` 쿼리문은 `<insert></insert>` 태그, `UPDATE` 쿼리는 `<update></update>` 태그, `DELETE` 쿼리는 `<delete></delete>` 태그 안에 작성한다.
- 쿼리문 작성을 위한 모든 태그에는 `id` 속성을 반드시 작성해야 한다. `id` 속성은 각 쿼리문을 식별할 수 있는 이름이기에 중복되지 않도록 작성한다.
- `<select></select>` 태그만이 유일하게 `resultType` 혹은 `resultMap` 속성을 반드시 가져야 한다.
- `resultType`과 `resultMap` 속성은 `SELECT` 쿼리 결과 조회되는 데이터를 자바의 어떤 자료형에 담아 올 것인지 지정하는 속성이다.
- 조회 결과가 없는 `INSERT`, `UPDATE`, `DELETE` 쿼리문 실행에는 `resultType`, `resultMap` 속성을 작성하지 않는다.
- `resultType`은 조회 쿼리 결과 데이터를 자바가 제공하는 자료형(`int`, `String`, `List`)으로 가져올 수 있을 때 사용한다.
- `resultMap`은 조회 쿼리 결과 데이터를 우리가 만든 자료형(`class`)으로 가져 올 때 사용한다.

다음 슬라이드에서 xml 파일에 조회 쿼리에 대한 몇가지 예시를 보도록 하자.

```
<select id="getReadCnt" resultType="int">
    SELECT READ_CNT
    FROM BASIC_BOARD
    WHERE BOARD_NUM = 1
</select>
```

	READ_CNT
1	3

- SELECT 쿼리문을 작성해야 하기에 <select> 태그를 사용하였다.
- id는 작성되는 여러 쿼리문을 구분하기 위한 식별자다. 중복이 없어야 한다.
- xml 파일에 작성하는 쿼리문의 끝에서 세미콜론(;)을 붙이든 말든 상관없지만, 안붙이는 것을 추천한다.
- 쿼리 실행 결과 조회되는 데이터를 자바로 가져오기 위해 작성한 것이 resultType="int" 코드다.
- 예시의 쿼리 실행 결과 조회되는 데이터는 숫자이기 때문에 자바의 int 자료형으로 조회된 데이터를 받을 수 있기 때문이다.
- 만약 여러분이 쿼리 실행결과 어떠한 데이터가 조회되는지 모른다면(쿼리 공부가 덜 됐다면,,) resultType 속성에 어떤 내용을 작성할지 모를 것이다. 그러니, 쿼리 공부하자.

```
<select id="getReadCntAll" resultType="int">
    SELECT READ_CNT
    FROM BASIC_BOARD
</select>
```

	READ_CNT
1	3
2	0
3	5

- 좌측 쿼리의 실행 결과 데이터도 자바의 int에 담을 수 있다. 그래서 resultType에 int를 작성하였다.
- 여기서 생각해봐야 할 것은 조회되는 행의 갯수이다.
- WHERE 조건이 없기 때문에 쿼리문을 실행하면 테이블에 저장된 데이터의 갯수만큼 READ_CNT가 여러개 조회된다. 자바의 int자료형은 하나의 정수만 저장 가능한 자료형이다. 그런데도 불구하고 조회되는 데이터를 자바로 가져오기 위해 resultType에 int를 작성했다.
- 결론은, resultType에 작성하는 자료형은 조회되는 모든 데이터를 자바로 가져오기 위한 자료형을 작성하는 것이 아니다. 조회되는 모든 데이터 중 하나의 행 데이터를 담을 수 있는 자료형을 작성하는 것이다.
- resultMap 속성 또한 조회되는 모든 데이터 중 하나의 행 데이터를 가져오기 위한 자료형을 작성한다.

```
<select id="getTitle" resultType="String">
    SELECT TITLE
    FROM BASIC_BOARD
    WHERE BOARD_NUM = 2
</select>
```

- 조회 결과 문자열 데이터가 조회되며, 문자열 데이터를 자바로 가져오기 위해서는 String에 담아 올 수 있다.
- 그렇기 때문에 resultType에 String을 작성하였다.

	TITLE
1	제목2

```
<select id="getTitleAll" resultType="String">
    SELECT TITLE
    FROM BASIC_BOARD
</select>
```

- 조회 결과 문자열 데이터가 테이블에 저장된 데이터의 갯수만큼 조회된다.
- resultType에는 String을 작성하였다.
- 다시 말하지만 resultType에 작성하는 자료형은 조회되는 모든 데이터를 자바로 가져오기 위한 자료형을 작성하는 것이 아니다. 조회되는 모든 데이터 중 하나의 행 데이터를 담을 수 있는 자료형을 작성하는 것이다.

	TITLE
1	제목1
2	제목2
3	제목3

다음의 쿼리 실행 결과 조회되는 데이터의 resultType은?

```
SELECT *
```

```
FROM BASIC_BOARD
```

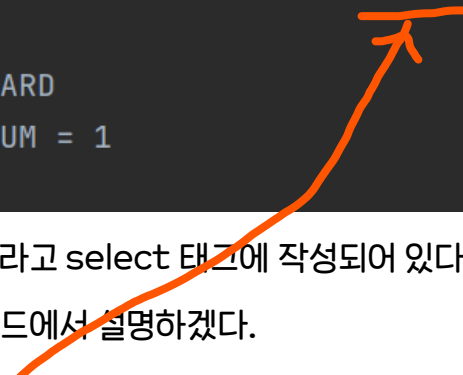
```
WHERE BOARD_NUM = 1;
```

	BOARD_NUM	TITLE	WRITER	CONTENT	READ_CNT	CREATE_DATE
1	1	제목1	Kim	제목1의 내용입니다	3	2025-02-01 23:45:49

- resultType 속성에는 조회되는 하나의 행 데이터를 자바로 가져올 때 어떤 자료형에 담아 가져올 것인지 작성한다고 거듭 말했다.
- 위와 같이 행 데이터가 조회되면 int에 담을수도, String에 담을 수도 없다. 애초에 이러한 데이터를 받을 수 없는 자료형은 자바에 존재하지 않는다.
- 하지만 자바는 필요한 자료형을 만들 수 있다. 그것이 class다. 우리는 위 쿼리의 실행 결과 조회되는 하나의 행 데이터를 가져올 수 있는 자료형을 이미 만들었다. 그것이 바로 BoardDTO 클래스인 것이다! 결국 위와 같은 쿼리 실행 결과 조회되는 한 행의 데이터는 BoardDTO 자료형에 담아 자바로 가져올 수 있다.
- 이렇듯, 데이터베이스에서 조회되는 데이터를 자바로 가져오기 위해 자바에서 제공하는 int, double, String 등이 아닌 우리가 만든 자료형(클래스)을 사용할 때는 resultType이 아니라, resultMap을 사용한다.

아래는 resultMap을 사용한 예시이다.

```
<select id="getBoardList" resultMap="board">
  SELECT *
  FROM BASIC_BOARD
  WHERE BOARD_NUM = 1
</select>
```



resultMap="board"라고 select 태그에 작성되어 있다. 하지만 resultMap 속성에 작성한 board가 무슨 의미인지는 아직 모를 것이다.

이것에 대해 다음 슬라이드에서 설명하겠다.

board-mapper.xml 파일의 mapper 태그 안에 다음과 같은 코드를 작성한다.

아래 코드를 보면 <resultMap></resultMap> 태그가 작성되어 있다. 그리고 해당 태그 안에는 <id>태그와 <result>태그가 사용되고 있다.

이 <resultMap> 태그는 데이터베이스에서 조회된 데이터를 자바의 어떤 자료형에 담아 올지 정의하고 있는 코드이다.

```
<mapper namespace="com.green.board.mapper.BoardMapper">
  <resultMap id="board" type="com.green.board.dto.BoardDTO">
    <id column="BOARD_NUM"          property="boardNum"/>
    <result column="TITLE"           property="title"/>
    <result column="WRITER"          property="writer"/>
    <result column="CONTENT"         property="content"/>
    <result column="READ_CNT"        property="readCnt"/>
    <result column="CREATE_DATE"     property="createDate"/>
  </resultMap>
```

<id>태그와 <result> 태그를 보면 column 속성과 property 속성이 사용된 것을 볼 수 있다.

여기서 column 속성에는 SELECT 쿼리 결과 조회되는 컬럼명을 작성하고, property에는 해당 컬럼의 값을 자바로 가져오는 변수명을 작성한다.

<resultMap> 태그의 id 속성은 해당 resultMap에 부여되는 식별자이고, type 속성은 조회된 데이터를 가져올 클래스명을 패키지명과 함께 작성한 것이다.

우리가 정의한 resultMap을
식별하기 위한 id 속성이다.

조회된 데이터를 받아올 클래스를
패키지명을 포함해서 작성한다.

```
<mapper namespace="com.green.board.mapper.BoardMapper">
  <resultMap id="board" type="com.green.board.dto.BoardDTO">
    <id column="BOARD_NUM" property="boardNum"/>
    <result column="TITLE" property="title"/>
    <result column="WRITER" property="writer"/>
    <result column="CONTENT" property="content"/>
    <result column="READ_CNT" property="readCnt"/>
    <result column="CREATE_DATE" property="createDate"/>
  </resultMap>
</mapper>
```

property 속성에는 조회된
데이터를 자바로 가져올
클래스의 멤버변수를 작성한다.

```
@Setter
@Getter
@ToString
public class BoardDTO {
    private int boardNum;
    private String title;
    private String writer;
    private String content;
    private int readCnt;
    private LocalDateTime createDate;
}
```

column 속성에는 SQL문에서 조회되는 컬럼명을 작
성한다. (pk컬럼은 id태그로 작성)

```
CREATE TABLE BASIC_BOARD (
    BOARD_NUM INT PRIMARY KEY AUTO_INCREMENT
    , TITLE VARCHAR(30)
    , WRITER VARCHAR(20)
    , CONTENT VARCHAR(50)
    , READ_CNT INT DEFAULT 0
    , CREATE_DATE DATETIME DEFAULT SYSDATE()
);
```

작성된 resultMap을 해석하면 다음과 같다.

BOARD_NUM 컬럼이 조회되면 BoardDTO 클래스의 boardNum 변수에 데이터를 받아오겠다.

TITLE 컬럼이 조회되면 BoardDTO 클래스의 title 변수에 데이터를 받아오겠다.

WRITER컬럼이 조회되면 BoardDTO 클래스의 writer 변수에 받아오겠다...

이러한 방식으로 디비에서 조회된 데이터를 받아오는 resultMap을 board라고 부르겠다.

마지막으로 작성한 resultMap과 select 쿼리를 함께 살펴보면

```
<mapper namespace="com.green.board.mapper.BoardMapper">
  <resultMap id="board" type="com.green.board.dto.BoardDTO">
    <id column="BOARD_NUM"      property="boardNum"/>
    <result column="TITLE"       property="title"/>
    <result column="WRITER"      property="writer"/>
    <result column="CONTENT"     property="content"/>
    <result column="READ_CNT"    property="readCnt"/>
    <result column="CREATE_DATE" property="createDate"/>
  </resultMap>
```

```
<select id="getBoardList" resultMap="board">
  SELECT *
  FROM BASIC_BOARD
  WHERE BOARD_NUM = 1
</select>
```

<resultMap>태그의 id 속성과 <select>태그의 resultMap 속성이 같은 것을 확인할 수 있다.

결국, <select>태그에 작성한 쿼리문 실행 결과 조회되는 데이터는 자바로 가져올 때 board 라는 resultMap의 정의에 맞게 가져오겠다는 의미가 된다!

resultType은 조회되는 데이터 전체를 자바로 어떻게 가져올 것인가가 아니라 조회되는 데이터 중 한 행의 데이터를 어떻게 받아 올지 작성한다고 하였다.

resultMap도 resultType과 마찬가지로 조회되는 모든 데이터를 받아오는 자료형을 정의하는 것이 아닌, 한 행의 데이터를 받아오는 자료형을 작성한 것이다!

그렇기 때문에 아래처럼 여러 행이 조회되는 쿼리 데이터도 resultMap에 board를 작성한 것을 확인할 수 있다.

```
<select id="getBoardList" resultMap="board">
  SELECT *
  FROM BASIC_BOARD
</select>
```

```
<select id="getList2" resultMap="board">
    SELECT BOARD_NUM, TITLE, WRITER
    FROM BASIC_BOARD
</select>
```

	BOARD_NUM	TITLE	WRITER
1	1	제목1	Kim
2	2	제목2	Lee
3	3	제목3	Park

- 위 쿼리의 조회 결과 데이터도 자바로 가져 올 때 BoardDTO 자료형으로 받아 올 수 있다.
- 다만, 조회 결과 데이터를 받은 BoardDTO 객체에는 조회되는 데이터만 담기 때문에 content, readCnt, createDate 멤버변수는 default 값을 가진다.

```
<select id="getList2" resultMap="board">
    SELECT BOARD_NUM, TITLE AS SUBJECT, WRITER
    FROM BASIC_BOARD
</select>
```

	BOARD_NUM	SUBJECT	WRITER
1	1	제목1	Kim
2	2	제목2	Lee
3	3	제목3	Park

좌측의 조회 쿼리 결과 데이터를 좌측의 TestDTO 클래스 자료형으로 자바로 가져올 수 있을까?
결론은 조회되는 BOARD_NUM 컬럼의 값은 boardNum 멤버변수에,
조회되는 WRITER 컬럼의 값은 writer 멤버변수에 담겨 자바로 가져오지만,
조회되는 TITLE 컬럼의 값은 가져오지 못한다.

왜 TITLE 컬럼의 조회 데이터는 가져오지 못하는지 resultMap의 정의를 잘 생각해보자.

xml 파일에 쿼리를 작성하는 방법을 대략 알아봤으니, 실제 게시글 목록 데이터를 조회하는 쿼리문을 작성해보자.

아래와 같은 html 결과를 보여주기 위해서 쿼리문은 다음과 같이 작성한다.

No	제 목	작성자	작성일	조회수
6	집에 가고 싶다 T T	홍길동	2024-05-27	5
5	오늘 점심은 중식입니다.	홍길동	2024-05-27	5
4	컴퓨터 포맷 방법 알려주세요.	홍길동	2024-05-27	5
3	우리집 강아지는요...	홍길동	2024-05-27	5
2	이게 맞는거임?	홍길동	2024-05-27	5
1	첫 가입 인사 드려요.	홍길동	2024-05-27	5

```
<select id="selectBoardList" resultMap="board">
    SELECT
        BOARD_NUM
        , TITLE
        , WRITER
        , CREATE_DATE
        , READ_CNT
    FROM BASIC_BOARD
</select>
```

- 기본적으로 어떤 컬럼을 조회할지에 대한 결정은 **화면에 어떤 데이터를 보여주냐에 따라 달라진다.** 내용은 게시글 목록에는 필요없으니 조회 xxx
- 게시글 목록에 보면 제목, 작성자, 작성일, 조회수 정보가 화면에 보여야 한다. 그래서 해당 정보를 조회한 것이다.
- 참고로 표에 보이는 No는 글번호가 아니다. 단순히 행 번호이다. 결론적으로 이야기하면 게시글 목록 화면에는 글번호 데이터가 불필요하다. 하지만 쿼리문을 보면 글번호 데이터를 조회한 것을 확인할 수 있다. **조회 시, 화면에 보일 필요가 없더라도 PK컬럼은 무조건 조회하는 것이 좋다.**
- 게시글 목록이란 존재하는 모든 게시글 정보를 화면에 나타내야 하기에 쿼리문에 보면 WHERE 절이 없는 것을 확인할 수 있다.

이제 본격적으로 interface 파일에 작성할 내용에 대해 알아보겠다.

mapper interface 파일에는 xml에 작성한 쿼리문을 실행시킬 추상메서드를 정의한다.

쿼리를 실행시킬 추상메서드를 작성할 때는 메서드명, 메서드의 리턴타입, 메서드의 매개변수를 적절하게 작성해야 한다.

익숙해지면 메서드의 리턴타입과 매개변수를 자유자재로 작성할 수 있겠지만, 당연하게도 처음에는 쉽지 않을 것이다. 그렇기 때문에 익숙해지기 전까지는 아래의 내용을 토대로 추상메서드를 작성하길 바라며, mapper interface에 작성해야 하는 코드의 기본적인 내용을 먼저 살펴본 다음, 게시글 목록 조회 쿼리 실행을 위한 추상메서드를 작성해보겠다.

- mapper interface에 작성하는 추상 메서드명은 반드시 xml에 작성한 쿼리의 id 속성과 일치해야한다.
- 추상메서드의 매개변수는 작성한 쿼리문에서 변수의 값을 채우는 용도로 사용된다. 그렇기 때문에, 매개변수는 작성한 쿼리문에 따라 유동적으로 달라진다.
- 추상메서드의 리턴타입은 쿼리 실행 후 결과 데이터를 모두 받아 올 수 있는 자료형으로 선언한다. 결국, 쿼리 실행 결과 데이터를 예상하지 못한다면 추상 메서드의 리턴타입을 작성 할 수 없다.
- xml 파일의 <select> 태그에 resultType, resultMap 속성도 조회한 데이터를 자바의 어떤 자료형에 받아올지 작성하는 것이라 하였다. 그럼 interface에 작성하는 추상메서드와의 리턴타입과 다른점은 무엇인가? xml 파일에 작성하는 resultType, resultMap은 조회되는 데이터 중 하나의 행 데이터를 어떤 자료형으로 받아올지 작성한다. interfac에 선언하는 추상메서드의 리턴타입은 조회한 전체 데이터를 어떤 자료형에 받아올지 작성한다!

위의 내용을 보면 알겠지만 추상메서드의 리턴타입, 매개변수를 작성하려면 쿼리를 작성할 수 있어야 하며, 쿼리의 실행 결과도 예측할 수 있어야 한다.

쿼리문이 모든 코드 작성의 기준이기 때문에 쿼리문은 정말 중요하다.

다음 슬라이드에서는 위에서 설명한 추상메서드 작성에 대한 몇 가지 예시를 보도록 하자.

```
<select id="getReadCnt" resultType="int">
    SELECT READ_CNT
    FROM BASIC_BOARD
    WHERE BOARD_NUM = 1
</select>
```

```
public int getReadCnt();
```

- 왼쪽과 같은 쿼리문을 실행시킬 추상메서드는 public int getReadCnt();로 선언하면 된다.
- 추상메서드명은 실행할 쿼리문의 id 속성과 반드시 동일하게 작성해야 한다.
- 이번 쿼리문의 매개변수는 필요없다. 매개변수는 작성한 쿼리문의 변수값을 채워주는 역할을 한다.
- 해당 쿼리문에는 채워줘야 할 변수가 없기 때문에 매개변수를 작성하지 않는다.
- 추상메서드의 리턴타입은 쿼리 실행 결과 데이터를 자바로 받아 올 수 있는 자료형으로 선언한다. 해당 쿼리문을 실행하면 숫자 데이터가 한 행 조회된다. 해당 데이터는 int 자료형에 담아 올 수 있기 때문에 리턴타입이 int다.

```
<select id="getBoardNum" resultType="int">
    SELECT BOARD_NUM
    FROM BASIC_BOARD
    WHERE READ_CNT > #{readCnt}
</select>
```

```
public List<Integer> getBoardNum(int readCnt);
```

쿼리 결과 조회되는 행 갯수를 생각할 때는 테이블에 현재 저장된 데이터로만 판단하는 것이 아니다. 테이블에 데이터가 많다고 가정하고 조회되는 행 갯수를 판단해야 한다.

- 왼쪽의 쿼리문을 실행시킬 추상메서드는 public List<Integer> getBoardNum(int readCnt); 이다.
- 추상메서드명은 실행할 쿼리문의 id 속성과 반드시 동일하게 작성해야 한다.
- 작성한 쿼리문을 보면 #{readCnt} 이라고 작성한 부분이 있다. 이렇게 #{ }로 표현된 부분이 쿼리 실행 시 채워줘야 할 변수다.
- #{ } 안에 작성하는 채워줘야 할 변수의 이름(여기서는 readCnt로 작성)은 컬럼명에 따라 작성한다.
- 해당 쿼리문이 실행되기 위해서는 readCnt 값을 채워야 한다. readCnt는 숫자 데이터기 때문에, 결국 해당 쿼리문이 실행되려면 숫자 데이터 하나를 매개변수로 가져와서 채워야 한다.
- 추상메서드의 리턴타입은 쿼리 실행 결과 데이터를 자바로 받아 올 수 있는 자료형으로 선언한다. 해당 쿼리문을 실행하면 숫자 데이터가 여러 행 조회된다. 정수 여러개를 담을 수 있는 자바의 자료형은 List<Integer>이기 때문에 추상메서드의 리턴타입에는 List<Integer>를 작성한다.

```
<select id="getBoardTitle" resultType="String">
    SELECT TITLE
    FROM BASIC_BOARD
    WHERE WRITER = #{writer}
</select>
```

```
public List<String> getBoardTitle(String writer);
```

- 왼쪽의 쿼리문을 실행시킬 추상메서드는 `public List<String> getBoardTitle(String writer);`이다.
- 추상메서드명은 실행할 쿼리문의 id 속성과 반드시 동일하게 작성해야 한다.
- 이번 쿼리문에서는 `#{writer}`에 작성자 데이터를 채워줘야 한다. 그렇기 때문에 해당 빈 값을 채우기 위해 추상 메서드의 매개변수에 `String writer`를 작성하였다.
- 작성자 한 명이 여러 게시글을 작성할 수 있기 때문에 이번 쿼리의 실행결과는 여러 행의 제목이 조회된다. 그렇기 때문에 조회된 데이터를 자바로 가져오기 위한 리턴타입으로 `List<String>`을 작성하였다.

```
<select id="getBoard" resultMap="board" >
    SELECT *
    FROM BASIC_BOARD
    WHERE BOARD_NUM = #{boardNum}
</select>
```

```
public BoardDTO getBoard(int boardNum);
```

- 왼쪽의 쿼리문을 실행시킬 추상메서드는 `public BoardDTO getBoard(int boardNum);`이다.
- 추상메서드명은 실행할 쿼리문의 id 속성과 반드시 동일하게 작성해야 한다.
- 이번 쿼리문에서는 `#{boardNum}`에 글번호 데이터를 채워줘야 한다. 그렇기 때문에 해당 빈 값을 채우기 위해 추상 메서드의 매개변수에 `int boardNum`을 작성하였다.
- 글번호는 기본키 컬럼이기 때문에 이번 쿼리의 실행결과는 테이블에 데이터가 아무리 많다고 하더라도, 한 행만 조회된다. 그렇기 때문에 조회된 데이터를 자바로 가져오기 위한 리턴타입으로 `BoardDTO`를 작성하였다.

```
<select id="getBoard"      resultMap="board" >
    SELECT *
    FROM BASIC_BOARD
    WHERE WRITER = #{writer}
    AND READ_CNT = #{readCnt}
</select>
```

```
public List<BoardDTO> getBoard(BoardDTO boardDTO);
```

- 이번 쿼리문을 실행시킬 메서드는 public List<BoardDTO> getBoard(BoardDTO boardDTO);이다.
- 추상메서드명은 실행할 쿼리문의 id 속성과 반드시 동일하게 작성해야 한다.
- 이번 쿼리문에서는 채워줘야 할 데이터가 2개이다. 그럼 두 개의 값을 채워주기 위해 매개변수도 2개 사용할까? 그렇게 해도 되지만 우리 매개변수로 BoardDTO 객체 하나만 사용하면 된다. BoardDTO 객체 하나로 writer값과 readCnt값 2개를 모두 받아올 수 있기 때문이다.
- 작성자 한 명이 동일한 조회수를 지닌 여러 게시글을 작성할 수 있기 때문에 이번 쿼리의 실행결과는 여러 행이 조회된다. 그렇기 때문에 조회된 데이터를 자바로 가져오기 위한 리턴타입으로 List<BoardDTO>를 작성하였다.

```
<update id="updateBoard">
    UPDATE BASIC_BOARD
    SET
    TITLE = #{title},
    CONTENT = #{content}
    WHERE BOARD_NUM = #{boardNum}
</update>
```

```
public int updateBoard(BoardDTO boardDTO);
```

- 이번 쿼리문은 업데이트 쿼리이기 때문에 <update></update> 태그 안에 작성하였다.
- <select></select> 태그를 제외한 모든 태그에는 resultType 속성을 추가하지 않는다.
- 이번 쿼리문에는 채워줘야 할 값이 3개지만, BoardDTO 객체 하나에 3개의 값을 모두 받아 올 수 있기 때문에 매개변수에는 BoardDTO 객체 하나만 받아오면 된다.
- 추상메서드의 리턴타입에는 쿼리 결과 데이터를 받을 자료형을 작성한다고 하였다. update의 쿼리 결과는 무엇인가? select를 제외한 insert, update, delete 쿼리는 조회 쿼리가 아니기 때문에 조회 데이터가 없다. 그렇기 때문에 insert, update, delete 쿼리의 결과는 쿼리 실행으로 영향을 받은 행 갯수가 결과가 된다. 왼쪽의 쿼리 실행 결과 1개의 행 데이터가 쿼리의 영향을 받는다. 결국, 이 쿼리의 실행 결과 데이터는 1이 된다.
- 결론적으로 insert, update, delete 쿼리의 리턴타입은 모두 int를 작성하기 것이 정석이다.(void 가능)

지금까지 mapper interface에서 정의하는 추상메서드의 작성방법에 대해 알아보았다.

그럼 게시글 목록 조회 쿼리를 실행할 수 있는 추상 메서드를 작성해보겠다.

xml파일에 우리가 게시글 목록 조회를 위한 쿼리문을 다음과 같이 작성하였다.

```
<select id="selectBoardList" resultMap="board">
  SELECT
    BOARD_NUM
    , TITLE
    , WRITER
    , CREATE_DATE
    , READ_CNT
  FROM BASIC_BOARD
</select>
```

좌측의 쿼리문을 실행시킬 추상 메서드는 다음과 같이 정의할 수 있다.

```
//게시글 목록 조회
public List<BoardDTO> selectBoardList ();
```

- mapper interface에 작성하는 추상 메서드명은 반드시 xml에 작성한 쿼리의 id 속성과 일치해야한다.
- 추상메서드의 매개변수는 작성한 쿼리문에서 변수의 값을 채우는 용도로 사용된다. 이번 쿼리문은 #{ } 형식의 코드가 없기 때문에 채울 데이터가 없다. 따라서, 매개변수를 작성하지 않는다.
- 추상메서드의 리턴타입은 쿼리 실행 후 결과 데이터를 받아 올 수 있는 자료형으로 선언한다. 해당 쿼리는 조회 결과 여러 행의 모든 컬럼이 조회된다. 하나의 행 데이터는 BoardDTO 객체 하나에 담아 올 수 있으니, 여러 행 데이터를 담으려면 BoardDTO 객체가 여러개 있어야 한다. 그렇기 때문에 이번 쿼리의 리턴타입에는 조회된 모든 데이터를 담을 수 있는 List<BoardDTO>를 작성해야 한다.

2. xml파일에 쿼리 작성, 3. mapper interface에 쿼리를 실행시킬 추상메서드 정의까지 끝냈다면

3번째로는 Service 클래스에 핵심 기능(쿼리 실행 메서드)을 구현하는 것이다. 현재는 com.green.board.service 패키지에 선언된 BoardService 파일이다.

Spring에서 관제탑 역할을 하는 클래스를 Controller라고 부르듯, 프로젝트의 핵심 기능을 담고 있는 클래스를 Service라고 부른다.

웹 개발 프로젝트에 핵심 기능은 데이터베이스 기능(데이터 조회, 삽입, 수정, 삭제)을 의미한다. 우리 프로젝트에서는 com.green.board.service 패키지에 선언된 BoardService 클래스를 우리 프로젝트의 핵심 기능을 담고 있는 클래스로 표현하려 한다.

BoardService 인터페이스에 게시글 목록 조회 기능을 수행할 추상메서드를 정의했다면, 마지막으로 BoardServiceImpl 클래스에 BoardService 인터페이스의 추상메서드를 구현하면 된다.

```
@Service
public class BoardService {
    //쿼리 실행 메서드가 존재하는 mapper interface 객체 선언
    private BoardMapper boardMapper;

    //생성자 주입으로 boardMapper 객체 의존성 주입
    @Autowired
    public BoardService(BoardMapper boardMapper){
        this.boardMapper = boardMapper;
    }

    //게시글 목록 조회 메서드
    public List<BoardDTO> getBoardList(){
        //게시글 목록 조회 쿼리 실행
        List<BoardDTO> boardList = boardMapper.getBordList();
        return boardList;
    }
}
```

- 먼저 클래스 위에 @Service 어노테이션을 사용하여 BoardService 클래스에 대한 객체를 생성했다. 객체를 생성하는 이유는 추후 컨트롤러에서 객체를 사용해야 하기 때문이다.
- BoardMapper 인터페이스에 대한 객체를 하나 선언하고, 해당 객체를 생성자 주입 방식을 통해 의존성 주입 받았다.
- BoardMapper 인터페이스에 대한 객체를 의존성 주입 받은 이유는, BoardMapper 인터페이스 안에 실제 게시글 조회 쿼리를 실행하는 메서드를 정의하였으며, 해당 메서드를 사용해야 하기 때문이다. (메서드를 호출하는 문법은 객체명.메서드() 이다. 그러니 당연히 객체가 있어야겠죠?)
- boardMapper.getBoardList() 코드가 핵심이다.
이 코드는 BoardMapper 인터페이스에 정의한 게시글 목록 조회 쿼리를 실행하는 메서드를 호출한 것이다. 해당 메서드가 실행되면 DB에서 게시글 목록을 조회하여, 조회된 데이터를 List<BoardDTO> 타입으로 자바로 가져온다.
- 마지막 return에는 조회된 게시글 목록 데이터를 가진 boardList 객체를 리턴하고 있다.
우리는 이렇게 return된 게시글 목록 데이터를 받아서 사용할 수 있다.
- 실 사용은 추후 컨트롤러에서 살펴보도록 하겠다.

BoardService 인터페이스에 게시글 목록 조회 기능을 수행할 추상메서드를 정의했다면, 마지막으로 BoardServiceImpl 클래스에 BoardService 인터페이스의 추상메서드를 구현하면 된다.

```
@Controller
public class BoardController {
    private BoardService boardService;

    @Autowired
    public BoardController(BoardService boardService){
        this.boardService = boardService;
    }

    @GetMapping("/")
    public String goBoardListPage(Model model){
        //게시글 목록 조회
        List<BoardDTO> boardList = boardService.getBoardList();

        //조회한 목록 데이터를html로 전달
        model.addAttribute("boardList", boardList);

        //게시글 목록 html 응답
        return "board_list";
    }
}
```

- 먼저 클래스 위에 @Controller 어노테이션을 사용하여 BoardController 클래스에 대한 객체를 생성했다. 요청이 들어오면 생성된 컨트롤러 클래스의 객체를 스프링컨테이너가 사용한다.
- BoardService 클래스에 대한 객체를 하나 선언하고, 해당 객체를 생성자 주입 방식을 통해 의존성 주입 받았다.
- BoardService에 대한 객체를 의존성 주입 받은 이유는, BoardService에서 구현한 메서드를 호출 하기 위함이다.
- Controller에서 요청에 대한 응답 처리를 위한 메서드 goBoardListPage 메서드를 정의하였다.
- 해당 메서드는 게시글 목록 html을 응답한다. 이때, 게시글 목록 데이터를 html에 표현하기 위해 조회한 게시글 목록 데이터를 model에 실어 html에 전달한다.

게시글 상세 조회

게시글 상세 정보 조회를 위한 쿼리문을 board-mapper.xml파일에 작성하면 다음과 같다.

```
<select id="getBoard" resultType="BoardDTO">
    SELECT *
    FROM BASIC_BOARD
    WHERE BOARD_NUM = #{boardNum}
</select>
```

- 쿼리문을 보면 WHERE절에 BOARD_NUM 컬럼을 사용한 것을 볼 수 있다. 게시글 상세 조회는 게시글이 아무리 많아도 딱 1개의 게시글만을 조회해야 한다. 이렇게 데이터가 아무리 많아도 하나의 데이터를 식별할 수 있는 컬럼은 기본키 속성이 부여된 컬럼이다. 게시글 테이블의 기본키 컬럼이 BOARD_NUM이기 때문에 WHERE절에서 사용되었다.
- 글 번호를 어떤 값으로 지정하냐에 따라 상세 조회되는 게시글이 달라진다. 만약, WHERE절에 'BOARD_NUM = 1' 이라는 쿼리가 작성되면 해당 쿼리 실행 시 무조건 글 번호가 1번인 게시글만 조회된다. 'BOARD_NUM = 2' 라는 쿼리문은 무조건 2번 글에 대한 상세정보만 조회할 것이다. 우리가 몇번 글을 선택하냐에 따라 조회되는 정보가 달라져야 하기 때문에, 글 번호는 매번 달라져야 한다. 이렇게 쿼리문에서 변수처럼 달라지는 값은 #{ } 안에 작성한다. 'BOARD_NUM = #{boardNum}'에서 #{boardNum}에는 결국 우리가 전달하는 데이터에 따라 값이 바뀌는 부분을 표현한 것이다. 해당 쿼리가 정상적으로 실행되려면 #{boardNum}값을 채울 수 데이터가 쿼리 실행 시 가져와야 한다.
- #{ }안의 단어는 해당 컬럼과 매칭되는 변수명을 작성한다.(BoardDTO 클래스에서 데이터베이스 테이블 컬럼과 매칭되는 변수)
- 해당 쿼리가 실행되면 조회되는 행의 갯수는 무조건 하나이다. WHERE절에 기본키 컬럼을 사용하였기 때문이다. 현재 쿼리에서 조회되는 한 행의 데이터는 BoardDTO 자료형에 담아 자바로 가져올 수 있는 형태이다. 그렇기 때문에 resultType에는 BoardDTO 자료형을 지정하였다.

board-mapper.xml 파일에 게시글 상세 정보 조회 쿼리문을 작성하였다면, 다음으로 BoardMapper 인터페이스에 쿼리를 실행해 줄 추상메서드를 정의한다.

게시글 상세 정보를 조회하기 위한 추상메서드는 다음과 같이 작성하였다.

```
@Mapper
public interface BoardMapper {
    //게시글 목록 조회 쿼리 실행 메서드
    public List<BoardDTO> getBoardList();

    //게시글 상세 조회 쿼리 실행 메서드
    public BoardDTO getBoard(int boardNum);
}
```

```
<select id="getBoard" resultMap="board" >
    SELECT *
    FROM BASIC_BOARD
    WHERE BOARD_NUM = #{boardNum}
</select>
```

- 게시글 상세 정보 조회를 위한 쿼리문의 id를 xml 파일에서 getBoard로 작성하였다. BoardMapper 인터페이스에서 선언하는 메서드명은 반드시 쿼리 id와 일치해야하기 때문에 추상메서드명을 getBoard로 작성하였다.
- 추상메서드의 매개변수는 실행할 쿼리문의 빈 값을 채울 용도로 사용된다. 이번에 작성한 쿼리문을 보면 #{boardNum} 이라 작성된 부분에 글번호 데이터를 채워야 쿼리문이 정상적으로 실행된다. 채워야 하는 글번호는 자바의 자료형으로 판단하면 int형이기 때문에 매개변수의 자료형을 int형으로 선언한 것이다. 매개변수의 이름은 무엇이든 상관없으나, 코드상으로도 글 번호를 채워줄을 표현하기 위해 변수명으로 boardNum을 사용하였다.
- 추상메서드의 리턴타입은 쿼리 실행 후 결과 데이터를 자바로 가져오기 위한 자료형을 작성한다. 작성한 쿼리문을 실행하면 무조건 하나의 행만을 조회하며, 조회되는 하나의 행 데이터는 BoardDTO 자료형에 담아 자바로 가져올 수 있는 형태다. 그렇기 때문에 추상메서드의 리턴타입에서 BoardDTO를 선언한 것이다.

다음으로 BoardService 클래스에 BoardMapper 인터페이스에 정의한 추상메서드를 구현하면 된다.

```
@Mapper
public interface BoardMapper {
    //게시글 목록 조회 쿼리 실행 메서드
    public List<BoardDTO> getBoardList();

    //게시글 상세 조회 쿼리 실행 메서드
    public BoardDTO getBoard(int boardNum);
}
```

```
@Override
public BoardDTO getBoard(int boardNum) {
    System.out.println("게시글 상세 조회 기능 시작");
    BoardDTO board = boardMapper.getBoard(boardNum);
    System.out.println("게시글 상세 조회 기능 끝");
    return board;
}
```

- 위 코드에서 핵심코드는 BoardDTO board = boardMapper.getBoard(boardNum); 이다.
- boardMapper.getBoard(boardNum); 코드는 BoardMapper 인터페이스에서 정의한 게시글 상세 조회 쿼리 실행 메서드를 호출한 것이다.
- 이 코드가 실행되면 xml에서 정의한 게시글 상세 정보 조회 쿼리가 실행된다.
- 이때, 매개변수로 전달한 boardNum값이 실제 쿼리의 #{boardNum} 부분을 채우게 된다.
- BoardDTO board = boardMapper.getBoard(boardNum); 코드에서 '=' 기준 우측 코드가 실행되면, 매개변수로 전달된 글 번호를 갖는 게시글 상세 정보가 조회된다. 그리고 이렇게 조회된 게시글 상세 정보 데이터는 '=' 기준 좌측에 선언한 board 객체에 담기게 된다. board 객체는 조회된 게시글 상세 정보를 갖고 있는 객체이기에, 이 객체를 마지막에 return하면 우리가 받아와서 사용할 수 있다.

아래 코드는 BoardController 클래스에 작성한 게시글 상세 정보 조회 요청에 응답하는 메서드다.

```
@GetMapping("/boards/{boardNum}")
public String goBoardDetailPage(@PathVariable("boardNum") int boardNum, Model model){
    //게시글 상세 조회
    BoardDTO boardDetail = boardService.getBoard(boardNum);

    //조회한 게시글 상세 데이터를 html로 전달
    model.addAttribute("board", boardDetail);

    //게시글 상세 조회 html 응답
    return "board_detail";
}
```

- @GetMapping() 어노테이션을 보면 알겠지만, 상세 조회하려는 게시글의 글번호를 URL Parameter 방식으로 전달받는다.
- URL Parameter 로 글 번호가 1번이 전달되면, 해당 글번호를 갖는 게시글 정보를 조회한다. 글 번호 3번이 전달되면 3번 글에 대한 상세 정보를 조회한다.
- 조회한 데이터를 Model 인터페이스를 사용하여 html로 전달하였다.

게시글 등록

게시글 등록을 위한 쿼리문을 board-mapper.xml파일에 작성하면 다음과 같다.

```
<insert id="insertBoard">
    INSERT INTO BASIC_BOARD (
        TITLE
        , WRITER
        , CONTENT
    ) VALUES (
        #{title}
        , #{writer}
        , #{content}
    )
</insert>
```

- INSERT쿼리는 <insert></insert> 태그에 작성하며, resultType 속성을 갖지 않는다.
- id 속성의 값은 중복되지 않게 작성하며, id만 봐도 어떤 쿼리를 작성하는지 유추할 수 있도록 작성한다.
- INSERT 쿼리문의 컬럼을 보면 BASIC_BOARD 테이블에서 BOARD_NUM, READ_CNT, CREATE_DATE 컬럼은 제외된 것을 확인할 수 있다.
BOARD_NUM 컬럼은 AUTO_INCREMENT가 적용되어 있어, 제외해도 알아서 중복되지 않은 값을 넣어준다.
READ_CNT는 DEFAULT 0 이 적용되어, 제외해도 기본적으로 0값을 갖게 된다.
CREATE_DATE는 DEFAULT SYSDATE()가 적용되어, 제외해도 기본적으로 쿼리를 실행하는 시점의 날짜 및 시간 데이터가 들어간다.
- 쿼리문에서 #{ }로 표현한 코드가 데이터를 채워줘야 할 부분이다. 이번 쿼리문이 정상적으로 실행되기 위해서는 제목, 작성자, 글 내용 3개의 데이터를 받아와야 한다.

board-mapper.xml 파일에 게시글 등록 쿼리문을 작성하였다면, 다음으로 BoardMapper 인터페이스에 쿼리를 실행해 줄 추상메서드를 정의한다.

게시글 등록을 위한 추상메서드는 다음과 같이 작성하였다.

```
<insert id="insertBoard">
  INSERT INTO BASIC_BOARD (
    TITLE
    , WRITER
    , CONTENT
  ) VALUES (
    #{title}
    , #{writer}
    , #{content}
  )
</insert>
```

```
//게시글 등록 쿼리 실행 메서드
public int insertBoard(BoardDTO boardDTO);
```

- 게시글 등록 위한 쿼리문의 id를 xml 파일에서 insertBoard로 작성하였다. BoardMapper 인터페이스에서 선언하는 메서드명은 반드시 쿼리 id와 일치해야하기 때문에 추상메서드명을 insertBoard로 작성하였다.
- 추상메서드의 매개변수는 실행할 쿼리문의 빈 값을 채울 용도로 사용된다. 이번에 작성한 쿼리문을 보면 #{title}, #{writer}, #{content} 3개의 데이터를 채워야 쿼리문이 정상적으로 실행된다. 채워줘야 하는 3개의 데이터는 BoardDTO객체 하나에 다 담아 올 수 있기 때문에 BoardDTO 객체를 매개변수로 선언하였다.
- 추상메서드의 리턴타입은 쿼리 실행 후 결과 데이터를 자바로 가져오기 위한 자료형을 작성한다. INSERT 쿼리의 실행 결과 조회되는 데이터가 없다. 그렇기 때문에 INSERT 쿼리의 실행 결과는 쿼리 결과 영향을 받는 행 갯수가 된다. (지금 쿼리문의 영향을 받는 행의 갯수는 1이다. 하나의 행 데이터가 삽입되기 때문이다) 그렇기 때문에 영향을 받은 행 갯수를 담아 올 수 있는 int로 선언하였다.(update, delete 쿼리문도 영향을 받은 행 갯수가 결과이기 때문에 리턴 타입을 int로 사용한다)

다음은 BoardService 인터페이스에 게시글 등록 기능을 추상메서드로 작성한다.

BoardService 인터페이스에 작성하는 추상메서드는 BoardMapper 인터페이스에 작성한 코드와 동일하게 작성하면 된다.

```
@Mapper
public interface BoardMapper {
    //게시글 목록 조회 쿼리 실행 메서드
    public List<BoardDTO> getBoardList();

    //게시글 상세 조회 쿼리 실행 메서드
    public BoardDTO getBoard(int boardNum);

    //게시글 등록 쿼리 실행 메서드
    public int insertBoard(BoardDTO boardDTO);
}
```

```
//게시글 등록 메서드
public int insertBoard(BoardDTO boardDTO){
    //게시글 등록 쿼리 실행
    //삽입, 수정, 삭제 쿼리 결과는 쿼리 실행에 영향을 받은 행 갯수이기에 리턴타입을 int 지정!
    //삽입, 수정, 삭제 쿼리 실행 메서드의 리턴타입은 void도 사용한다.
    int resultRowCnt = boardMapper.insertBoard(boardDTO);
    return resultRowCnt;
}
```

- 위 코드에서 핵심코드는 `int resultRowCnt = boardMapper.insertBoard(boardDTO);` 이다.
- `boardMapper.insertBoard(boardDTO);` 코드는 BoardMapper 인터페이스에서 정의한 게시글 등록 쿼리 실행 메서드를 호출한 것이다.
- 이 코드가 실행되면 xml에서 정의한 게시글 등록 쿼리가 실행된다.
- 이때, 매개변수로 전달한 boardDTO 객체에 저장된 값이 실제 쿼리의 `#{title}`, `#{writer}`, `#{content}` 부분을 채우게 된다.
- `int resultRowCnt = boardMapper.insertBoard(boardDTO);` 코드에서 '=' 기준 우측 코드가 실행되면, 매개변수로 전달된 boardDTO 객체 안의 제목, 작성자, 글 내용을 BASIC_BOARD 테이블에 INSERT 한다. 그리고 INSERT 쿼리의 영향을 받은 행 갯수가 '=' 기준 좌측에 선언한 resultRowCnt 변수에 담기게 된다.
(여기서는 1행이 삽입되었기 때문에 resultRowCnt 변수에는 1이 반환된다.)

아래 코드는 BoardController 클래스에 작성한 게시글 등록 요청에 응답하는 메서드다.

```
//게시글 등록
@PostMapping("/insert")
public String insertBoard(BoardDTO boardDTO){
    //html에서 입력받은 데이터 확인
    System.out.println(boardDTO);

    //게시글 등록 쿼리 실행
    boardService.insertBoard(boardDTO);

    //게시글 등록 쿼리 실행 후 게시글 목록 페이지로 응답
    return "redirect:/";
}
```

- @PostMapping을 사용하였다. 그 말은 html에서 입력한 데이터를 form 태그로 전송한다는 것이다.
- 전달되는 데이터는 커맨드 객체를 이용하여 받았다.
- 쿼리 실행 메서드의 매개변수로 html에서 전달받은 모든 데이터가 들어있는 커맨드 객체를 전달하면, 쿼리 실행 시 빈 값을 채워 정상적으로 쿼리문이 실행된다.
- 게시글 등록 후 게시글 목록 페이지를 응답하기 위해 'redirect:'를 이용하였다.
- 컨트롤러 안에서 정의한 메서드의 return은 응답 html을 의미하지만, 만약 지금처럼 'redirect:' 키워드가 붙으면 html로 응답하는 것이 아니라, 컨트롤러의 메서드로 재요청을 보내는 것이다. 이렇게 재요청을 보내는 이유는 이미 게시글 목록 페이지로 응답하는 메서드를 만들었기 때문에, 중복 코드를 작성하지 않고 만들어놓은 메서드를 재사용하기 위해서다.